Aprendizaje Federado

Extensiones de Machine Learning

Ginés Carrillo Ibáñez
Yago Ibarrola Lapeña
Aarón Palomar Peña

Universidad de Murcia
Facultad de Informática

Curso 2025/2026

Contenidos

1 Sesión 1

Para esta sesión, junto con la instalación de Nébula, se han realizado los siguientes cuatro experimentos sobre el entrenamiento de una CNN en el dataset CIFAR-10 de clasificación de imágenes. Se han realizado 10 rondas de entrenamiento en una única época, se han desplegado 4 nodos y las actualizaciones se han calculado usando FedAvg. Los experimentos realizados han sido:

- Entrenamiento centralizado con datos homogéneos (IID) y heterogéneos (non-IID) entre los nodos.
- Entrenamiento descentralizado con datos homogéneos (IID) y heterogéneos (non-IID) entre los nodos.

1.1 FL centralizado

Comparamos el entrenamiento IID y non-IID entre los nodos. En el caso non-IID, se utiliza una distribución de dirichlet con $\alpha = 0.1$. Se puede ver la comparación de ambos casos en la Figura **??**:

IID **non-IID**

Figure 1: Comparación del entrenamiento centralizado con datos IID y non-IID.

Vemos que en el caso de datos homogéneos, el modelo converge más rápidamente y alcanza una mayor precisión, llegando en la ronda 10 a un F1-score de aproximadamente 0.65, mientras que en non-IID solo alcanza un F1-score de 0.47. Esto tiene sentido, ya que en el caso non-IID, cada nodo tiene una distribución de datos diferente, lo que dificulta la generalización del modelo global. Igualmente, en la ronda 3 IID ya alcanza un F1-score de 0.52, mientras que non-IID solo llega a 0.3.

1.2 FL descentralizado

En este caso, se ha realizado el mismo experimento pero con un entrenamiento descentralizado. Se muestran los resultados en la Figura **??**:

IID **non-IID**

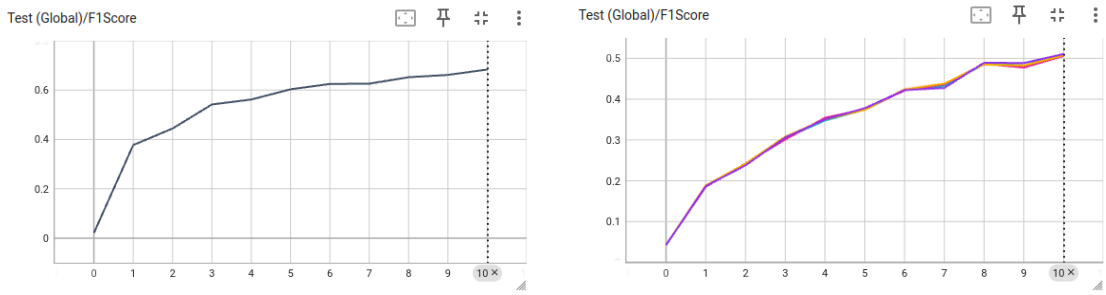


Figure 2: Comparación del entrenamiento descentralizado con datos IID y non-IID.

- Ronda 3: IID alcanza un F1-score de 0.5, mientras que non-IID solo llega a 0.3.
- Ronda 10: IID alcanza un F1-score de aproximadamente 0.67, mientras que non-IID solo alcanza un F1-score de 0.5.

En general, aunque CFL y DFL muestran resultados similares en términos de rendimiento (F1-score), presentan diferencias importantes a nivel de arquitectura. CFL destaca por su simplicidad y menor sobrecarga de red y recursos, pero introduce un punto único de fallo y posibles cuellos de botella en federaciones grandes. Por otro lado, DFL ofrece mayor resiliencia frente a fallos y ataques, a costa de

una mayor complejidad en la gestión de modelos, sincronización para la agregación y comunicaciones entre nodos.

Podemos ver las matrices de confusión para ambos casos en la Figura **??**:

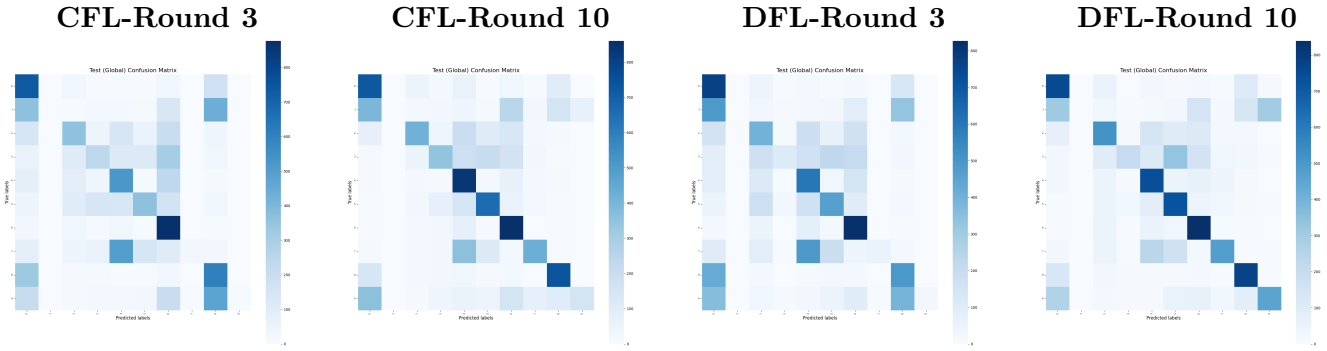


Figure 3: Matrices de confusión para el entrenamiento non-IID en CFL y DFL en las rondas 3 y 10.

Vemos que ambas rondas 3 son muy parecidas, donde todavía la diagonal no está muy marcada, tendiendo a predecir las clases 0, 4 y 8. En la ronda 10 sí que se aprecian menos errores, con una diagonal más marcada.

2 Sesión 2

Para esta sesión, se ha utilizado de nuevo la configuracion de entrenamiento de DFL non-IID de la sesión anterior. Se han realizado con ella los siguientes experimentos:

- Ataque de *model poisoning* con 10% de ruido gaussiano.
- Ataque de *label flipping* con 80% de muestras envenenadas.

Para cada uno de los ataques, compararemos con un 20% y un 50% de clientes maliciosos.

2.1 Model poisoning

En este ataque, los clientes maliciosos añaden ruido gaussiano a sus modelos locales antes de enviarlos para la agregación. Este ruido degrada la calidad de las actualizaciones, afectando al proceso de convergencia.

Los resultados obtenidos se muestran en la Figura **??**:

20% nodos maliciosos **50% nodos maliciosos**

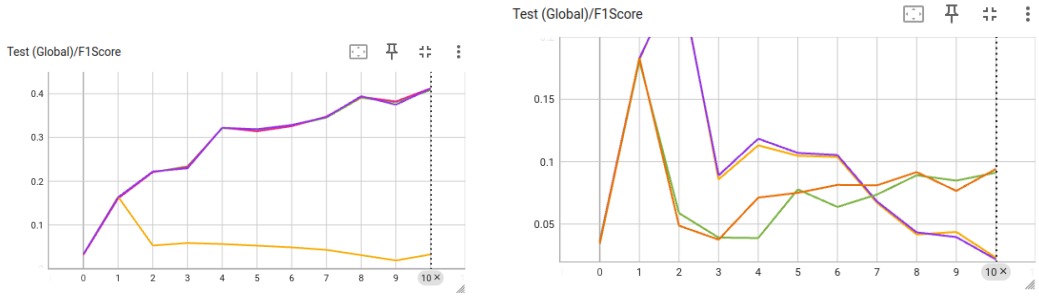


Figure 4: Impacto del ataque de *model poisoning* con diferentes porcentajes de nodos maliciosos.

Se observa que el impacto del ataque aumenta significativamente con el número de nodos maliciosos, provocando una notable inestabilidad:

- Con un 20% de nodos maliciosos, en un primer momento el modelo parece converger, pero a partir de la ronda 3, el rendimiento se desploma, alcanzando un F1-score final de apenas 0.08.
- Con un 50% de nodos maliciosos, el comportamiento es similar, pero la caída es aún más pronunciada, pues los dos nodos benignos acaban con un F1-score cercano a 0, ya que al haber la mitad de nodos maliciosos ha sido imposible para los modelos aprender nada útil y se han visto arrastrados por el ruido introducido.

2.2 Label flipping

En este caso, los nodos maliciosos alteran las etiquetas de un 80% de sus muestras de entrenamiento de manera aleatoria. Este ataque introduce información incorrecta directamente en el proceso de aprendizaje.

Los resultados se muestran en la Figura **??**:

20% nodos maliciosos **50% nodos maliciosos**

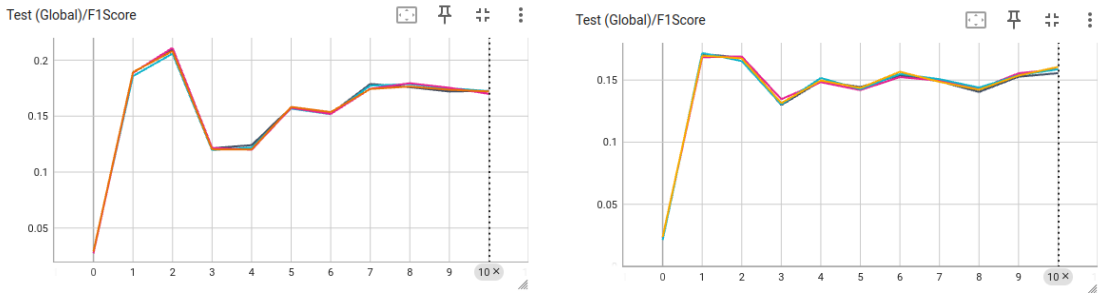


Figure 5: Impacto del ataque de *label flipping* con diferentes porcentajes de nodos maliciosos.

En este escenario, el impacto reduce el rendimiento pero mantiene un entrenamiento más estable:

- Con un 20% de nodos maliciosos, el modelo presenta una convergencia similar para todos los clientes, pero el rendimiento se desploma, alcanzando un F1-score final en torno a 0.17.
- Con un 50% de nodos maliciosos, el modelo logra converger estancándose de manera uniforme en un F1-score de 0.16.

2.3 Comparación y análisis

Comparando ambos ataques y configuraciones con los datos obtenidos, se extraen las siguientes conclusiones:

- Contrario a lo que se podría intuir en un inicio, el ataque de *model poisoning* de 10% genera un escenario más inestable con dos nodos maliciosos que el de *label flipping* al 80%. Mientras el primero reduce la convergencia (F1-score 0.08), el segundo logra engañar al modelo llevándolo a una convergencia falsa pero ligeramente superior (F1-score 0.16).
- A medida que aumenta el porcentaje de clientes maliciosos (20% a 50%), la métrica final cae en ambos ataques: de 0.07 a 0.02 en *model poisoning*, y de 0.17 a 0.16 en *label flipping*.
- En la sesión anterior, el entorno DFL ideal (IID) alcanzaba un F1-score de 0.67, mientras que la simple introducción de heterogeneidad en los datos (DFL non-IID base) lo reducía a 0.50. Al introducir condiciones de ataque sobre este último escenario, la degradación se amplifica: en la condición de ataque más leve (20% *label flipping*), el F1-score ya cae por debajo del caso base a 0.17. En las condiciones de ataque más severas (50% *model poisoning*), el modelo se desploma hasta un 0.02. Esto evidencia que los entornos non-IID son sensibles ante actualizaciones maliciosas, ya que a la agregación le resulta mucho más difícil distinguir entre una variación legítima (datos sesgados) y un ataque intencionado.

3 Sesión 3

Para esta sesión, se ha utilizado de nuevo la configuración de entrenamiento DFL non-IID con la presencia de un nodo malicioso realizando *model poisoning*, tal y como se definió en la sesión anterior. Sobre este escenario, se han evaluado dos mecanismos de agregación robusta con el objetivo de mitigar el impacto de actualizaciones adversarias:

- Agregación mediante *Trimmed Mean*.
- Agregación mediante *Krum*.

3.1 Trimmed Mean

El algoritmo *Trimmed Mean* consiste en eliminar un porcentaje de los valores más extremos (potencialmente maliciosos) antes de realizar la agregación de los modelos, reduciendo así la influencia de outliers. En *Nebula*, hay que configurar el parámetro `beta` para indicar qué cantidad de valores extremos se desea eliminar. Por defecto, este valor es 0, lo que implicaba que se estaba utilizando una media simple sin recorte. Tras cambiar este valor a 1, los resultados muestran una mejora significativa, como se muestra en la Figura **??**.

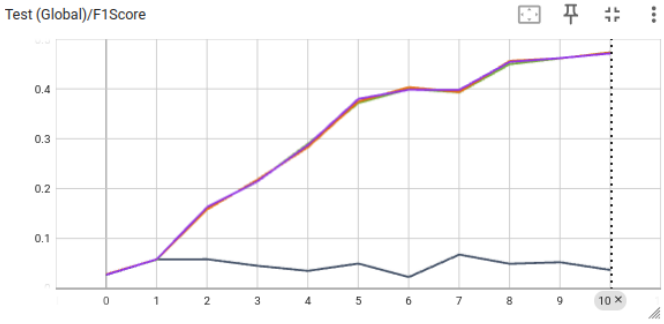


Figure 6: Resultados del entrenamiento con agregación *Trimmed Mean*.

Se observa que este método consigue anular prácticamente en su totalidad el efecto del nodo malicioso:

- El modelo converge de manera estable, evitando las oscilaciones bruscas observadas en *model poisoning* sin defensa.
- El F1-score final se sitúa en torno a 0.47, lo que supone una mejora significativa frente al caso atacado sin mecanismos de defensa (0.08).
- Sin embargo, el rendimiento sigue siendo ligeramente inferior al escenario sin ataques, indicando que el filtrado de valores extremos no elimina completamente el impacto adversario.

3.2 Krum

El algoritmo *Krum* selecciona la actualización más representativa entre los nodos, basándose en la distancia entre modelos, con el objetivo de descartar aquellos que se alejan significativamente (potencialmente maliciosos).

Los resultados se presentan en la Figura **??**:

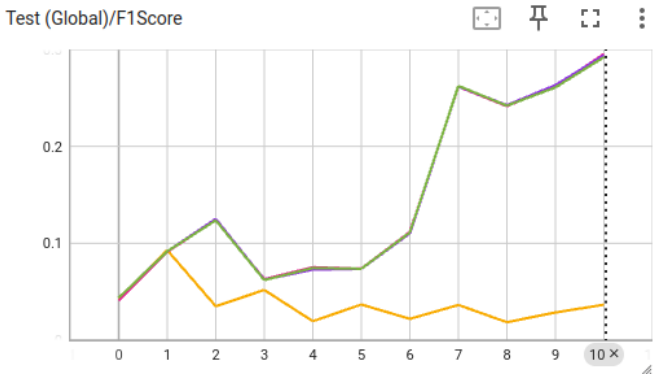


Figure 7: Resultados del entrenamiento con agregación *Krum*.

En este caso, el comportamiento es diferente:

- El entrenamiento mejora respecto al escenario sin defensa, pero no alcanza la estabilidad observada con *Trimmed Mean*.
- En un primer momento, parece que a los nodos benignos les cuesta distinguir al nodo malicioso, lo que provoca un comportamiento aleatorio hasta la ronda 6, donde ya empieza a converger.
- El F1-score final se sitúa aproximadamente en 0.29, lejos de los valores obtenidos con *Trimmed Mean*.
- Esto sugiere que, aunque Krum es capaz de descartar actualizaciones maliciosas, puede verse afectado por la heterogeneidad non-IID, seleccionando modelos que no representan adecuadamente la distribución global.

Vemos la matriz de confusión tanto del nodo malicioso como de uno de los nodos benignos en la última ronda, en la Figura **??**:

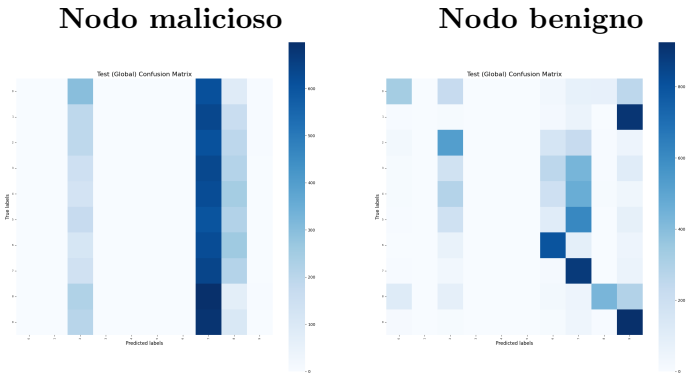


Figure 8: Matrices de confusión del nodo malicioso y uno de los nodos benignos en la última ronda con agregación Krum.

Vemos que el modelo del nodo malicoso predice únicamente las clases 2, 7 y 8. Esto hace que el nodo benigno se vea arrastrado, haciendo que tanto la clase 2 como la clase 7 se vean sobrepredichas y sin embargo hay clases como la 3, 4 y 5 que no tienen ningún verdadero positivo, porque se confunden tanto con la clase 2 como con la clase 7.

3.3 Comparación y análisis

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- *Trimmed Mean* demuestra una mayor robustez frente a ataques de *model poisoning*, ya que consigue estabilizar la convergencia y alcanzar un F1-score superior en promedio.
- En escenarios con datos heterogéneos, *Krum* puede verse penalizado, ya que la variabilidad entre nodos puede confundirse con comportamiento malicioso, afectando negativamente a la selección de modelos.
- Ambos métodos mejoran claramente el rendimiento respecto al escenario atacado sin defensa (F1-score 0.08), pero no alcanzan los niveles del escenario sin ataques. Esto confirma que, aunque las técnicas robustas ayudan, no eliminan completamente el efecto adversarial.

4 Sesión 4

4.1 Pesos estáticos

Las reputaciones calculadas por cada nodo se muestran en la Figura **??**:

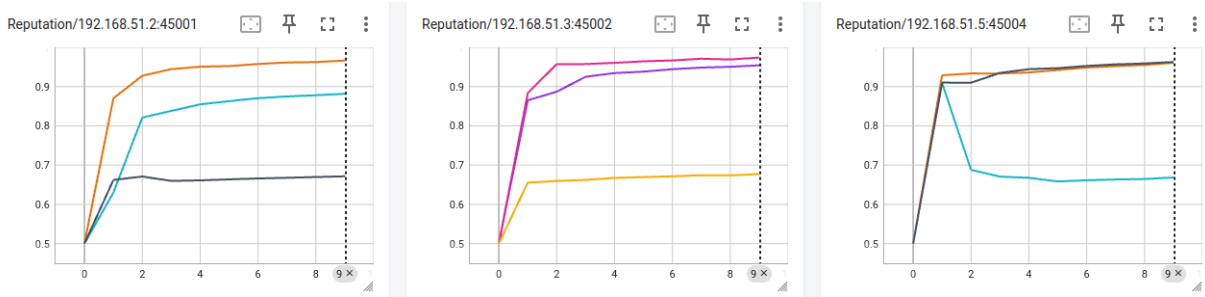
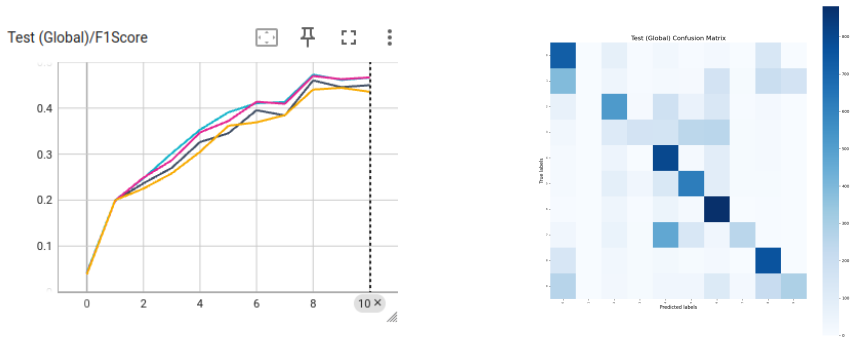


Figure 9: Reputación con pesos estáticos a lo largo del entrenamiento.

Vemos que los tres nodos honestos otorgan una reputación inferior al nodo malicioso. Desde el punto de vista de cada nodo honesto, los demás nodos honestos tienen una reputación muy cercana a 1, mientras que la reputación del nodo malicioso es algo inferior a 0.7. Esta diferencia coincide con la poderación de la métrica de número de mensajes (0.3), en la que el nodo malicioso obtiene una puntuación terrible (envía alrededor de 15 mensajes por ronda). Dado que el nodo malicioso obtiene una buena puntuación en las otras métricas, la reputación total no es tan baja como cabría esperar.

En cuanto a los resultados del entrenamiento con esta configuración, se muestran en la Figura **??**:



images/reputation_static_c

Figure 10: Resultados del entrenamiento con pesos estáticos: F1-score y matrices de confusión. Matriz de confusión del nodo malicioso a la derecha.

La gráfica del F1-score es errática: el nodo malicioso alcanza un rendimiento comparable al de un nodo honesto, mientras que otro nodo honesto presenta una alta variabilidad, y otro tiene un rendimiento muy bajo. En cuanto a las matrices de confusión, la del nodo malicioso (derecha) muestra una mejor clasificación frente al nodo honesto (izquierda). Esto sugiere que el sistema de reputación con pesos estáticos no ha sido capaz de aislar al nodo malicioso.

4.2 Pesos dinámicos

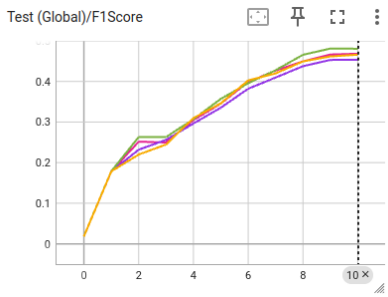
En esta configuración, los pesos se ajustan dinámicamente para cada métrica en función de las condiciones del entorno.

Los resultados se presentan en la Figura **??**:

images/reputation_dynamic.png

Figure 11: Reputación con pesos dinámicos a lo largo del entrenamiento.

En este caso, se observa que los nodos honestos asignan una reputación mucho más baja al nodo malicioso y de manera más rápida. Los resultados del entrenamiento con esta configuración se muestran en la Figura **??**:



images/reputation_dynamic_cm2.png images/reputation_dynamic_

Figure 12: Resultados del entrenamiento con pesos dinámicos: F1-score y matrices de confusión. Matriz de confusión del nodo distinto (no malicioso) a la derecha.

La gráfica de F1-score es ahora más estable, pero muestra un resultado curioso. Tres nodos alcanzan un F1-score similar, cercano a 0.35, mientras que el nodo restante (que no es el malicioso) tiene un rendimiento muy inferior. En cuanto a las matrices de confusión, la de la izquierda es una de los tres nodos con buen

rendimiento (que tienen matrices de confusión casi idénticas), mientras que la de la derecha corresponde al nodo con peor rendimiento. Para interpretar estos resultados, observamos que al inicio del entrenamiento, el nodo 0 (que es el que obtiene peor rendimiento) comienza creyendo que hay dos nodos maliciosos en la federación, lo que causa que descarte las actualizaciones de un nodo honesto. Esto, sumado a que el nodo 0 cuenta con un dataset mucho más pequeño que el resto, hace que obtenga un modelo global bastante inferior.

4.3 Comparación y análisis

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- La reputación con *pesos dinámicos* demuestra una mayor eficacia frente a ataques de *flooding*, y cabe esperar que también sea superior a *pesos estáticos* frente a otros tipos de ataques ya que consigue alcanzar un F1-score superior en promedio.
- Ambos métodos presentan limitaciones para aislar completamente al nodo malicioso, y al menos un nodo honesto se ve fuertemente afectado por el ataque en ambos escenarios.
- Curiosamente, el nodo malicioso ha estado en ambos escenarios entre los que mejores resultados encuentran. Esto se debe a que, a pesar de que en la agregación no se tienen en cuenta las actualizaciones de modelos con poca reputación, sí que se les sigue enviando el modelo global actualizado. Esto permite que el nodo malicioso siga aprendiendo y mejorando su rendimiento. Además, el nodo malicioso es el único que utiliza sus datos propios para entrenar, lo que le aporta cierta ventaja adicional.